



Figure 6: Self-verification examples. We only display the partial prompt for brevity. See Appendix, Sec. A.5 for the full prompt structure.

the task’s completion, at which point we add this new skill to the skill library and ask the automatic curriculum for a new objective (Fig. 2). If the agent gets stuck after 4 rounds of code generation, then we query the curriculum for another task. This iterative prompting approach significantly improves program synthesis for embodied control, enabling VOYAGER to continuously acquire diverse skills without human intervention.

3 Experiments

3.1 Experimental Setup

We leverage OpenAI’s gpt-4-0314 [35] and gpt-3.5-turbo-0301 [50] APIs for text completion, along with text-embedding-ada-002 [51] API for text embedding. We set all temperatures to 0 except for the automatic curriculum, which uses temperature = 0.1 to encourage task diversity. Our simulation environment is built on top of MineDojo [23] and leverages Mineflayer [52] JavaScript APIs for motor controls. See Appendix, Sec. B.1 for more details.

3.2 Baselines

Because there is no LLM-based agents that work out of the box for Minecraft, we make our best effort to select a number of representative algorithms as baselines. These methods are originally designed only for NLP tasks without embodiment, therefore we have to re-interpret them to be executable in MineDojo and compatible with our experimental setting:

ReAct [29] uses chain-of-thought prompting [46] by generating both reasoning traces and action plans with LLMs. We provide it with our environment feedback and the agent states as observations.

Reflexion [30] is built on top of ReAct [29] with self-reflection to infer more intuitive future actions. We provide it with execution errors and our self-verification module.

AutoGPT [28] is a popular software tool that automates NLP tasks by decomposing a high-level goal into multiple subgoals and executing them in a ReAct-style loop. We re-implement AutoGPT by using GPT-4 to do task decomposition and provide it with the agent states, environment feedback, and execution errors as observations for subgoal execution. Compared with VOYAGER, AutoGPT lacks the skill library for accumulating knowledge, self-verification for assessing task success, and automatic curriculum for open-ended exploration.

Note that we do not directly compare with prior methods that take Minecraft screen pixels as input and output low-level controls [53–55]. It would not be an apple-to-apple comparison, because we rely on the high-level Mineflayer [52] API to control the agent. Our work’s focus is on pushing the limits of GPT-4 for lifelong embodied agent learning, rather than solving the 3D perception or sensorimotor control problems. VOYAGER is orthogonal and can be combined with gradient-based approaches like

Table 1: Tech tree mastery. Fractions indicate the number of successful trials out of three total runs. 0/3 means the method fails to unlock a level of the tech tree within the maximal prompting iterations (160). Numbers are prompting iterations averaged over three trials. The fewer the iterations, the more efficient the method.

Method	Wooden Tool	Stone Tool	Iron Tool	Diamond Tool
ReAct [29]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
Reflexion [30]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
AutoGPT [28]	92 ± 72 (3/3)	94 ± 72 (3/3)	135 ± 103 (3/3)	N/A (0/3)
VOYAGER w/o Skill Library	7 ± 2 (3/3)	9 ± 4 (3/3)	29 ± 11 (3/3)	N/A (0/3)
VOYAGER (Ours)	6 ± 2 (3/3)	11 ± 2 (3/3)	21 ± 7 (3/3)	102 (1/3)

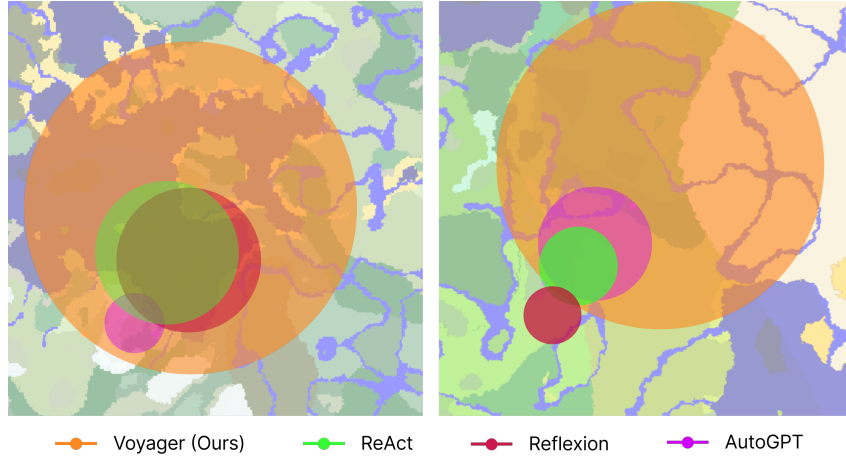


Figure 7: Map coverage: bird’s eye views of Minecraft maps. VOYAGER is able to traverse 2.3× longer distances compared to baselines while crossing diverse terrains.

VPT [8] as long as the controller provides a code API. We make a system-level comparison between VOYAGER and prior Minecraft agents in Table. A.2.

3.3 Evaluation Results

We systematically evaluate VOYAGER and baselines on their exploration performance, tech tree mastery, map coverage, and zero-shot generalization capability to novel tasks in a new world.

Significantly better exploration. Results of exploration performance are shown in Fig. 1. VOYAGER’s superiority is evident in its ability to consistently make new strides, discovering 63 unique items within 160 prompting iterations, 3.3× many novel items compared to its counterparts. On the other hand, AutoGPT lags considerably in discovering new items, while ReAct and Reflexion struggle to make significant progress, given the abstract nature of the open-ended exploration goal that is challenging to execute without an appropriate curriculum.

Consistent tech tree mastery. The Minecraft tech tree tests the agent’s ability to craft and use a hierarchy of tools. Progressing through this tree (wooden tool → stone tool → iron tool → diamond tool) requires the agent to master systematic and compositional skills. Compared with baselines, VOYAGER unlocks the wooden level 15.3× faster (in terms of the prompting iterations), the stone level 8.5× faster, the iron level 6.4× faster, and VOYAGER is the only one to unlock the diamond level of the tech tree (Fig. 2 and Table. 1). This underscores the effectiveness of the automatic curriculum, which consistently presents challenges of suitable complexity to facilitate the agent’s progress.

Extensive map traversal. VOYAGER is able to navigate distances 2.3× longer compared to baselines by traversing a variety of terrains, while the baseline agents often find themselves confined to local areas, which significantly hampers their capacity to discover new knowledge (Fig. 7).

Table 2: Zero-shot generalization to unseen tasks. Fractions indicate the number of successful trials out of three total attempts. 0/3 means the method fails to solve the task within the maximal prompting iterations (50). Numbers are prompting iterations averaged over three trials. The fewer the iterations, the more efficient the method.

Method	Diamond Pickaxe	Golden Sword	Lava Bucket	Compass
ReAct [29]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
Reflexion [30]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
AutoGPT [28]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
AutoGPT [28] w/ Our Skill Library	39 (1/3)	30 (1/3)	N/A (0/3)	30 (2/3)
VOYAGER w/o Skill Library	36 (2/3)	30 $\pm$ 9 (3/3)	27 $\pm$ 9 (3/3)	26 $\pm$ 3 (3/3)
VOYAGER (Ours)	19 $\pm$ 3 (3/3)	18 $\pm$ 7 (3/3)	21 $\pm$ 5 (3/3)	18 $\pm$ 2 (3/3)

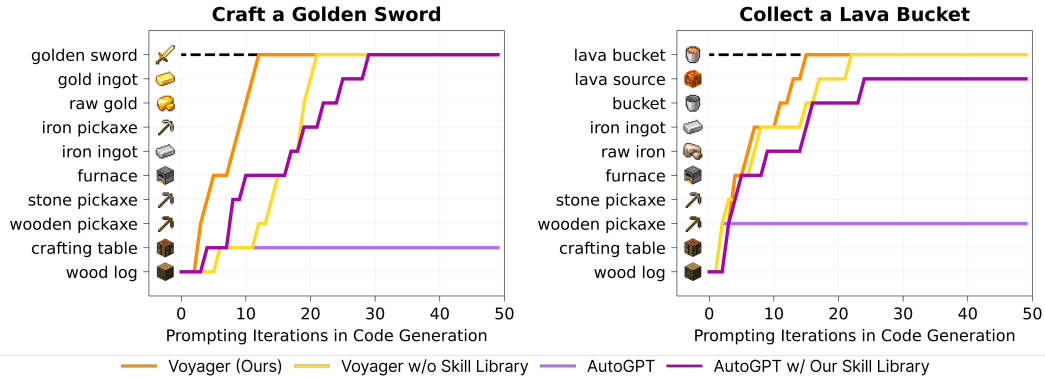


Figure 8: Zero-shot generalization to unseen tasks. We visualize the intermediate progress of each method on two tasks. See Appendix, Sec. B.4.3 for the other two tasks. We do not plot ReAct and Reflexion since they do not make any meaningful progress.

Efficient zero-shot generalization to unseen tasks. To evaluate zero-shot generalization, we clear the agent’s inventory, reset it to a newly instantiated world, and test it with unseen tasks. For both VOYAGER and AutoGPT, we utilize GPT-4 to break down the task into a series of subgoals. Table. 2 and Fig. 8 show VOYAGER can consistently solve all the tasks, while baselines cannot solve any task within 50 prompting iterations. What’s interesting to note is that our skill library constructed from lifelong learning not only enhances VOYAGER’s performance but also gives a boost to AutoGPT. This demonstrates that the skill library serves as a versatile tool that can be readily employed by other methods, effectively acting as a plug-and-play asset to enhance performance.

3.4 Ablation Studies

We ablate 6 design choices (automatic curriculum, skill library, environment feedback, execution errors, self-verification, and GPT-4 for code generation) in VOYAGER and study their impact on exploration performance (see Appendix, Sec. B.3 for details of each ablated variant). Results are shown in Fig. 9. We highlight the key findings below:

- **Automatic curriculum is crucial for the agent’s consistent progress.** The discovered item count drops by 93% if the curriculum is replaced with a random one, because certain tasks may be too challenging if attempted out of order. On the other hand, a manually designed curriculum requires significant Minecraft-specific expertise, and does not take into account the agent’s live situation. It falls short in the experimental results compared to our automatic curriculum.
- **VOYAGER w/o skill library exhibits a tendency to plateau in the later stages.** This underscores the pivotal role that the skill library plays in VOYAGER. It helps create more complex actions and steadily pushes the agent’s boundaries by encouraging new skills to be built upon older ones.

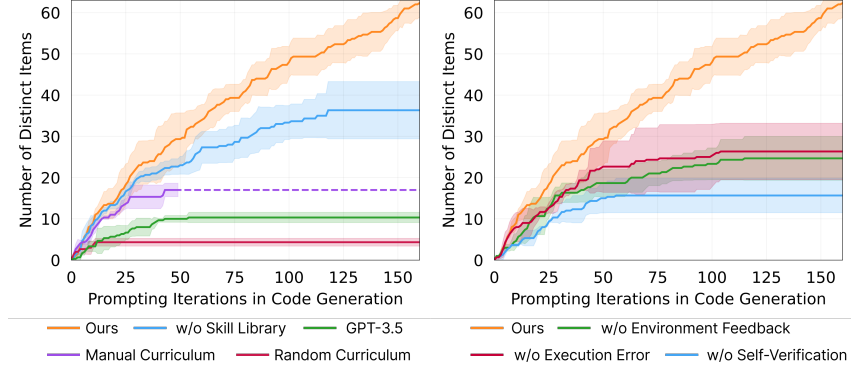


Figure 9: **Left: Ablation studies for the automatic curriculum, skill library, and GPT-4.** GPT-3.5 means replacing GPT-4 with GPT-3.5 for code generation. VOYAGER outperforms all the alternatives, demonstrating the critical role of each component. **Right: Ablation studies for the iterative prompting mechanism.** VOYAGER surpasses all the other options, thereby highlighting the essential significance of each type of feedback in the iterative prompting mechanism.

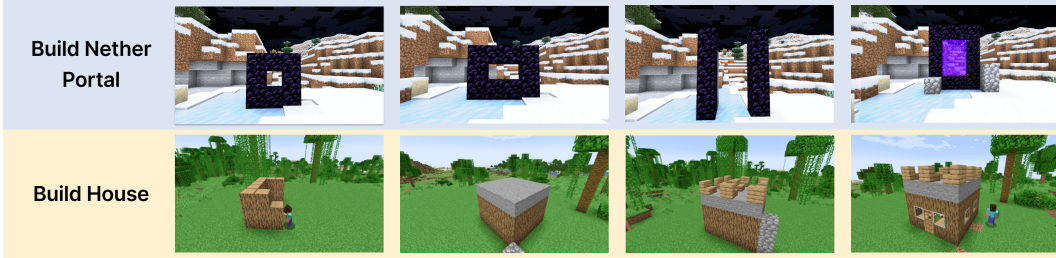


Figure 10: VOYAGER builds 3D structures with human feedback. The progress of building designs that integrate human input is demonstrated from left to right.

- **Self-verification is the most important among all the feedback types.** Removing the module leads to a significant drop (-73%) in the discovered item count. Self-verification serves as a critical mechanism to decide when to move on to a new task or reattempt a previously unsuccessful task.
- **GPT-4 significantly outperforms GPT-3.5 in code generation** and obtains $5.7\times$ more unique items, as GPT-4 exhibits a quantum leap in coding abilities. This finding corroborates recent studies in the literature [56, 57].

3.5 Multimodal Feedback from Humans

VOYAGER does not currently support visual perception, because the available version of GPT-4 API is text-only at the time of this writing. However, VOYAGER has the potential to be augmented by multimodal perception models [58, 59] to achieve more impressive tasks. We demonstrate that given human feedback, VOYAGER is able to construct complex 3D structures in Minecraft, such as a Nether Portal and a house (Fig. 10). There are two ways to integrate human feedback:

- (1) Human as a critic (equivalent to VOYAGER’s self-verification module): humans provide visual critique to VOYAGER, allowing it to modify the code from the previous round. This feedback is essential for correcting certain errors in the spatial details of a 3D structure that VOYAGER cannot perceive directly.
- (2) Human as a curriculum (equivalent to VOYAGER’s automatic curriculum module): humans break down a complex building task into smaller steps, guiding VOYAGER to complete them incrementally. This approach improves VOYAGER’s ability to handle more sophisticated 3D construction tasks.

4 Limitations and Future Work

Cost. The GPT-4 API incurs significant costs. It is $15\times$ more expensive than GPT-3.5. Nevertheless, VOYAGER requires the quantum leap in code generation quality from GPT-4 (Fig. 9), which GPT-3.5 and open-source LLMs cannot provide [60].

Inaccuracies. Despite the iterative prompting mechanism, there are still cases where the agent gets stuck and fails to generate the correct skill. The automatic curriculum has the flexibility to reattempt this task at a later time. Occasionally, self-verification module may also fail, such as not recognizing spider string as a success signal of beating a spider.

Hallucinations. The automatic curriculum occasionally proposes unachievable tasks. For example, it may ask the agent to craft a “copper sword” or “copper chestplate”, which are items that do not exist within the game. Hallucinations also occur during the code generation process. For instance, GPT-4 tends to use cobblestone as a fuel input, despite being an invalid fuel source in the game. Additionally, it may call functions absent in the provided control primitive APIs, leading to code execution errors.

We are confident that improvements in the GPT API models as well as novel techniques for finetuning open-source LLMs will overcome these limitations in the future.

5 Related work

Decision-making Agents in Minecraft. Minecraft is an open-ended 3D world with incredibly flexible game mechanics supporting a broad spectrum of activities. Built upon notable Minecraft benchmarks [23, 61–65], Minecraft learning algorithms can be divided into two categories: 1) Low-level controller: Many prior efforts leverage hierarchical reinforcement learning to learn from human demonstrations [66–68]. Kanitscheider et al. [14] design a curriculum based on success rates, but its objectives are limited to curated items. MineDojo [23] and VPT [8] utilize YouTube videos for large-scale pre-training. DreamerV3 [69], on the other hand, learns a world model to explore the environment and collect diamonds. 2) High-level planner: Vulum et al. [70] leverage few-shot prompting with Codex [41] to generate executable policies, but they require additional human interaction. Recent works leverage LLMs as a high-level planner in Minecraft by decomposing a high-level task into several subgoals following Minecraft recipes [55, 53, 71], thus lacking full exploration flexibility. Like these latter works, VOYAGER also uses LLMs as a high-level planner by prompting GPT-4 and utilizes Mineflayer [52] as a low-level controller following Vulum et al. [70]. Unlike prior works, VOYAGER employs an automatic curriculum that unfolds in a bottom-up manner, driven by curiosity, and therefore enables open-ended exploration.

Large Language Models for Agent Planning. Inspired by the strong emergent capabilities of LLMs, such as zero-shot prompting and complex reasoning [72, 37, 38, 36, 73, 74], embodied agent research [75–78] has witnessed a significant increase in the utilization of LLMs for planning purposes. Recent efforts can be roughly classified into two groups. 1) Large language models for robot learning: Many prior works apply LLMs to generate subgoals for robot planning [27, 27, 25, 79, 80]. Inner Monologue [26] incorporates environment feedback for robot planning with LLMs. Code as Policies [16] and ProgPrompt [22] directly leverage LLMs to generate executable robot policies. VIMA [19] and PaLM-E [59] fine-tune pre-trained LLMs to support multimodal prompts. 2) Large language models for text agents: ReAct [29] leverages chain-of-thought prompting [46] and generates both reasoning traces and task-specific actions with LLMs. Reflexion [30] is built upon ReAct [29] with self-reflection to enhance reasoning. AutoGPT [28] is a popular tool that automates NLP tasks by crafting a curriculum of multiple subgoals for completing a high-level goal while incorporating ReAct [29]’s reasoning and acting loops. DERA [81] frames a task as a dialogue between two GPT-4 [35] agents. Generative Agents [82] leverages ChatGPT [50] to simulate human behaviors by storing agents’ experiences as memories and retrieving those for planning, but its agent actions are not executable. SPRING [83] is a concurrent work that uses GPT-4 to extract game mechanics from game manuals, based on which it answers questions arranged in a directed acyclic graph and predicts the next action. All these works lack a skill library for developing more complex behaviors, which are crucial components for the success of VOYAGER in lifelong learning.

Code Generation with Execution. Code generation has been a longstanding challenge in NLP [41, 84, 85, 73, 37], with various works leveraging execution results to improve program